七、加密解密

詹文翰 高级实验师
zhanwenhan@163.com

课程安排


1
需求分析
2
系统设计
3
程序框架
4
软件测试
5
打包解包
6
压缩解压
7
加密解密
8
图形界面
9
软件发布
10
项目演示

实现功能

加密解密：指定密码对备份文件加密保存，支持多种类型的加密解密算法

加解密算法

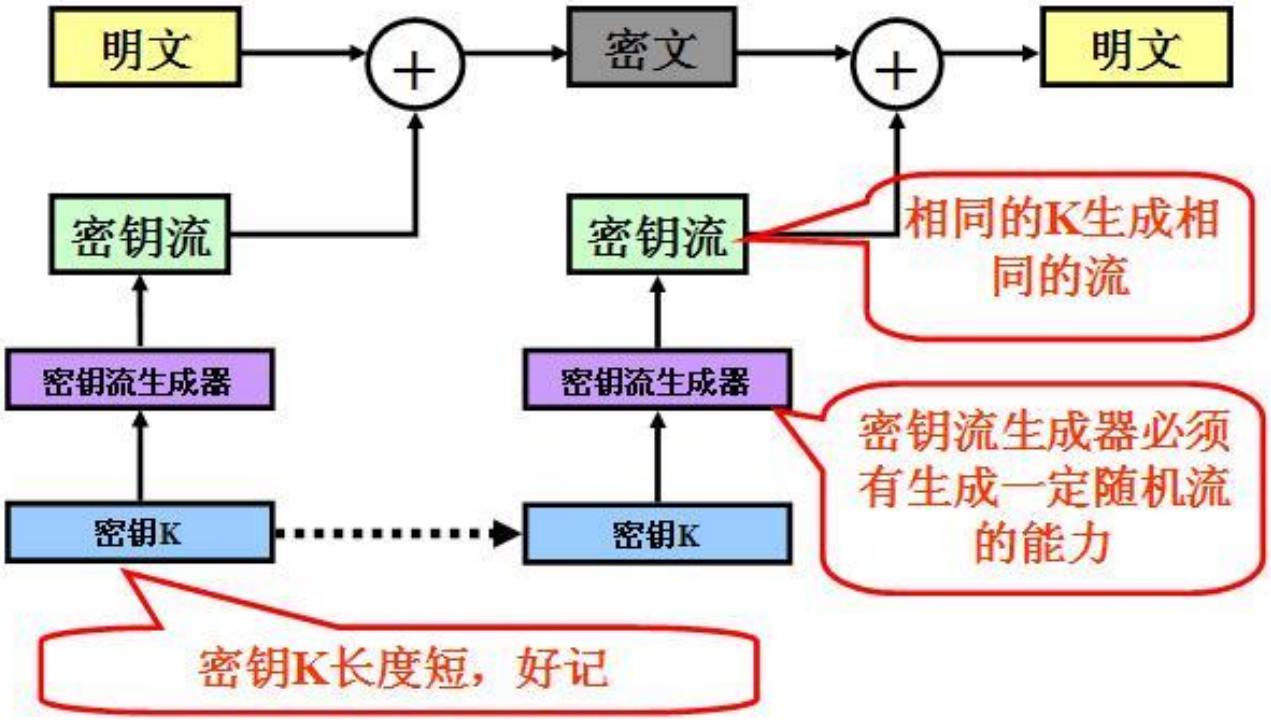
对于数据备份而言，主要考虑对称加密和散列算法。

散列算法也称为摘要算法，如MD5、SHA1等。

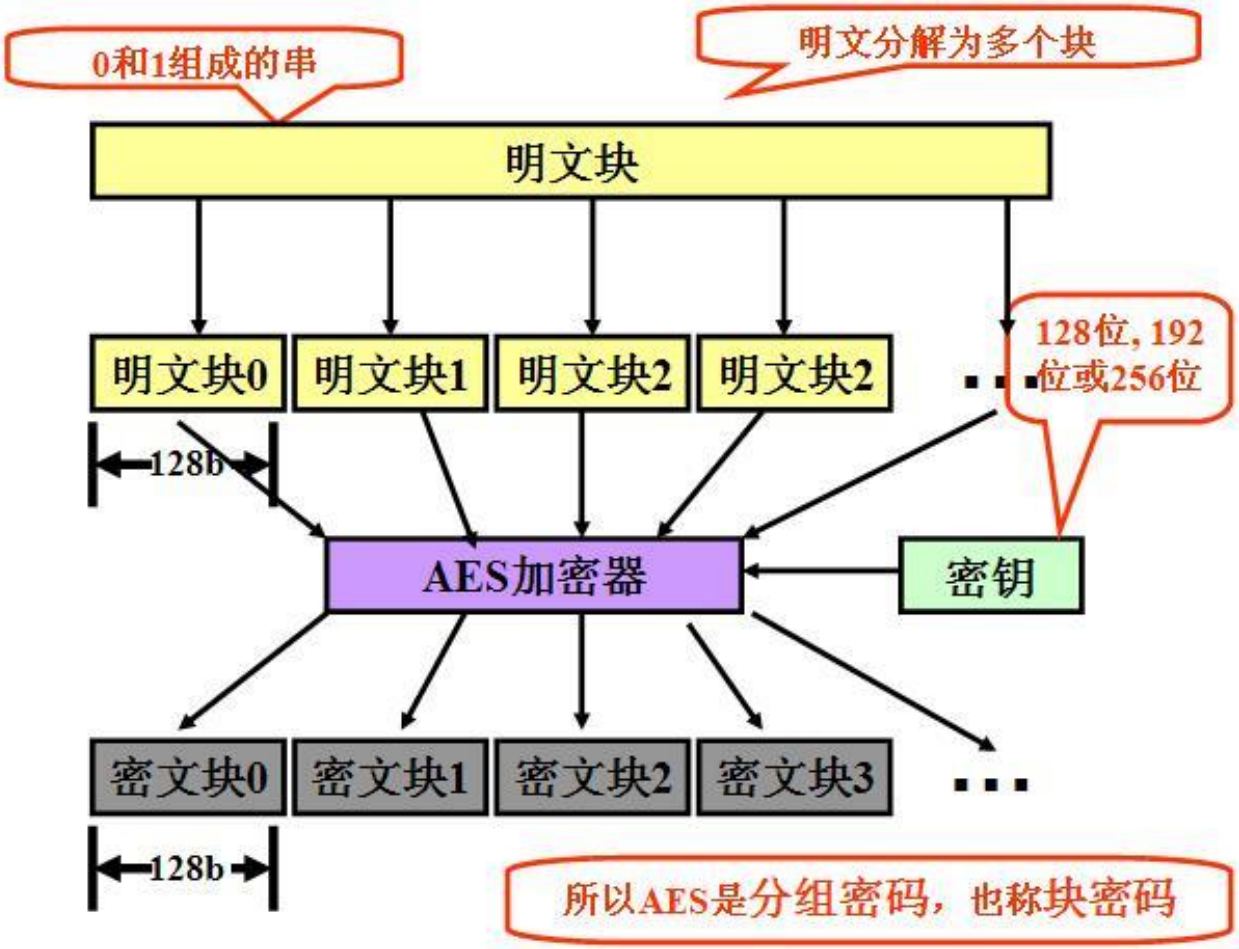
简单对称加密方式可考虑流密码。

更优的对称加密方式为分组加密，如DES、AES等。

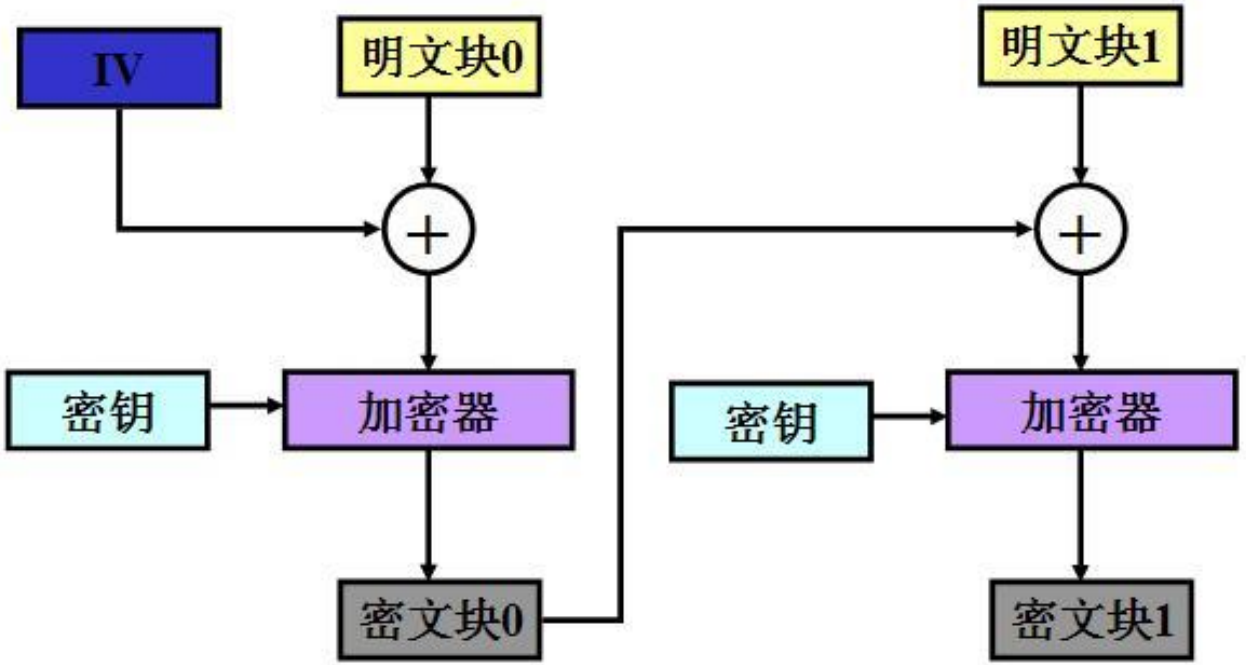
流密码


明文
+
密文
+
明文
密钥流
密钥流
相同的K生成相同的流
密钥流生成器
密钥流生成器
密钥流生成器必须有生成一定随机流的能力
密钥K
密钥K
密钥K长度短，好记

AES加密


0和1组成的串
明文分解为多个块
明文块
明文块0
明文块1
明文块2
明文块2
128位, 192位或256位
128b
AES加密器
密钥
密文块0
密文块1
密文块2
密文块3
128b
所以AES是分组密码，也称块密码

AES加密-CBC模式



这种模式称为CBC模式，又密码分组链接

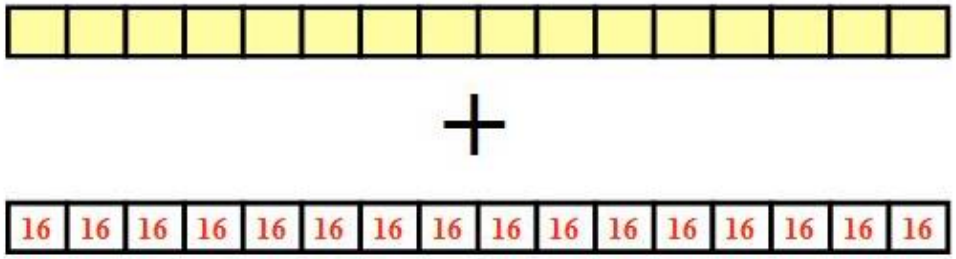
分组加密中的块补齐问题

一个AES块为16个字节（128位），需要考虑数据补齐问题。
主要分两种情况：

1、若数据的最后一块不够16个字节，则将最后一个数据块补齐为16个字节，填补的每个字符的内容为**填补字节的个数**。



2、若数据的最后一块刚好16个字节，则新增一块，该块中的每个字符的内容均为**块大小**，即16。



加解密常用库——OpenSSL

OpenSSL中的MD5、AES相关API：

MD5_Init
MD5_Update
MD5_Final
AES_set_encrypt_key
AES_set_decrypt_key
AES_ecb_encrypt
AES_cbc_encrypt
AES_cfb128_encrypt
AES_ofb128_encrypt

加解密常用库——OpenSSL

OpenSSL中的MD5、AES相关API：

MD5_Init
MD5_Update
MD5_Final
AES_set_encrypt_key
AES_set_decrypt_key
AES_ecb_encrypt
AES_cbc_encrypt
AES_cfb128_encrypt
AES_ofb128_encrypt

这些哈希和加密API在新版的OpenSSL手册中已经不再推荐使用。

推荐使用OpenSSL最新的EVP API。EVP API对哈希和加密算法进行了更高层的封装，屏蔽了算法之间使用上的细微差别。

加解密常用库——OpenSSL

OpenSSL EVP API中的常用函数：

EVP_MD_CTX_new
EVP_MD_CTX_free
EVP_MD_fetch
EVP_MD_free
EVP_DigestInit_ex
EVP_DigestUpdate
EVP_DigestFinal_ex

EVP_CIPHER_CTX_new
EVP_CIPHER_CTX_free
EVP_CIPHER_fetch
EVP_CIPHER_free
EVP_EncryptInit_ex
EVP_EncryptUpdate
EVP_EncryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
EVP_DecryptFinal_ex

加解密常用库——OpenSSL

OpenSSL EVP API中的常用函数：

EVP_MD_CTX_new
EVP_MD_CTX_free
EVP_MD_fetch
EVP_MD_free
EVP_DigestInit_ex
EVP_DigestUpdate
EVP_DigestFinal_ex

EVP_CIPHER_CTX_new
EVP_CIPHER_CTX_free
EVP_CIPHER_fetch
EVP_CIPHER_free
EVP_EncryptInit_ex
EVP_EncryptUpdate
EVP_EncryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
EVP_DecryptFinal_ex

官方手册：
https://docs.openssl.org/master/man3/

实验环节

完成以下实验内容：

- 通过调库方式完成本课程项目的加密解密功能
  - 至少完成两种加密解密算法的调用
- 采用测试驱动的开发方式（TDD）
  - 先编写测试用例
  - 再完成业务代码
  - 保证所有测试用例通过

参考加密解密代码

选修：构件实现

封装成库

封装成库

为什么要封装成库？

使程序的核心逻辑实现与界面实现分离开来

从而实现各自的技术栈和工作方向的独立

可以使用C++来暴露库接口吗？

可以，但并不推荐。

什么是ABI

ABI（Application Binary Interface）：应用二进制接口，是指不同程序或库之间的对接规范，以确保二进制可兼容性。ABI规范通常包含：

- 调用约定（函数调用及传参方式）
- 数据类型（格式和大小）
- 寄存器使用规则
- 堆/栈结构
- 名字装饰约定
- 错误和异常处理
- 模块初始化和终止过程

什么是ABI

不同的编程语言和平台有各自的ABI：

- Linux有IA32和x86-64 ABI
- C/C++ 在不同平台上有不同的ABI标准
- Java 和 .NET 均有各自的平台无关的ABI标准

C没有平台无关的ABI标准，各常用平台均有自己的C ABI标准，如：Linux、Windows、macOS、POSIX、ARM等。现行C++标准没有覆盖ABI，而是由各平台自行制定。

符合同一ABI的程序或库之间，可以在二进制级别互相调用和链接。
否则，则称之为二进制不兼容。

使用C暴露C++接口

在使用C++进行库开发时，对外暴露C接口通常更合适：

- C接口兼容性更好，很多语言都提供了调用C接口的机制。
- C接口没有C++的名称修饰问题（mangle）。
- C接口的ABI标准更为简单、通用。
- C接口不存在C++的异常传播问题。
- C接口受众更广。

参考接口和库的实现